JHQ-GPU: Representation-Driven GPU Execution and Workload-Adaptive Hierarchical Refinement

Rui Luo and
Mengxuan Zhang

Australian National University, Canberra, Australia
Rui.Luo1@anu.edu.au, mengxuan.zhang@anu.edu.au

Abstract. JHQ scores approximate nearest neighbours in two stages: a compact primary code over many candidates, then a per-dimension residual code over a small survivor set. Porting it to a GPU raises two questions that byte counting does not answer. First, how should the primary representation execute? Grouped lookup tables and the shared-memory-against-arithmetic trade-off they imply are established; what is not which of the two competing costs each grouping pays, or that JHQ's 256-entry table admits a grouping at all — its Cartesian codebook makes the table separate exactly into two 16-entry halves, an identity an ordinary product quantiser does not have. Second, how many survivors are worth refining? We select that budget with a label-free rule that compares the returned top-k sets at a candidate budget against a generous reference on sampled workload queries. On six datasets from 768 to 3,072 dimensions and up to 17.8M vectors, six controls contradict the naive byte-count prediction: a table-free form holds least state and loses 30–52%; four- and eight-way splits hold fewer entries than the two-way split and lose up to 61%; packed loading moves identical bytes and gains 1–66%, rising with probe depth. Disassembling the emitted kernel sharpens that: the granularity whose table does not fit issues the fewest instructions per candidate–subspace of any and is slower regardless, leaving residency as the term that has to account for it. At S=128 across four held-out configurations the rule is worth 1.095–1.469× steady-state, repaid within 4.0 to 19.2 batches of 500 queries, and its median choice matches the offline reference budget in all four; a quarter of that sample trades this for a quality risk that varies by workload. Against quantised IVF baselines JHQ owns the high-recall end at high dimension and indexes two datasets where the tested IVF-RaBitQ build path runs out of memory; graph-based CAGRA-int8 is 1.6–2.9× faster below Recall@10=0.95, at 3.9–7.4× the build time. The result is a bounded operating region, not a universal-fastest claim.

Keywords: Approximate nearest neighbour search · GPU · Quantisation · High-dimensional data · Adaptive query processing

1 Introduction

High-dimensional approximate nearest-neighbour (ANN) search is a core primitive in retrieval, recommendation and vector databases, and on a GPU compressed search is not governed by footprint alone: a representation can reduce bytes yet raise lookup pressure or instruction issue, and an implementation can move the same logical bytes and still accelerate. JHQ [3] makes that concrete because its hierarchy exposes two costs of different shape. After IVF routing a compact primary code scores every routed candidate at $O(n_c M)$; a per-dimension residual code is then read only for the c_k survivors of primary filtering, at $O(c_k d)$ with $c_k \ll n_c$. JHQ also leaves the refinement factor α in $c_k = \alpha k$ externally specified. The port therefore raises two coupled questions with different causes: *how should the primary representation execute*, and *how much of the expensive hierarchy should execute*?

Our answer to the first begins from prior art rather than against it. Grouping code bits into small tables and choosing the group size by trading on-chip capacity against arithmetic is established — GPU IVF-RaBitQ [8] does exactly this, at U=4 — and we claim none of it. Two things are not established. That JHQ admits the grouping at all: its 8-bit code indexes 256 codewords with no grouped structure on its face, and only because each codeword is a Cartesian product of one-dimensional levels does the table separate exactly into two 16-entry halves, an identity an ordinary k-means product quantiser does not have. And which cost each grouping pays: full-table state grows as $256M$ entries against $32M$ for the split, but each further split costs another lookup per candidate–subspace, and which term dominates is what we measure rather than assert. Our answer to the second is a label-free workload policy: sample queries from the running workload, compare the top-k sets they return at a candidate budget against a generous reference, and take the smallest budget that leaves them unchanged. It needs neither ground-truth neighbours nor a learned difficulty model, at the cost of assuming a reasonably stable query distribution.

We evaluate on six datasets from 768 to 3,072 dimensions, up to 17.8M vectors, on one RTX 5090, and we test the alternatives we rejected rather than only the implementation we kept. Six controls contradict the byte-count prediction, most sharply between the four- and eight-way splits, which hold the same number of entries and differ only in how many lookups reach them. Static disassembly then supplies the implicated variable without a profiler: issued instructions per candidate–subspace are 5.0, 13.0, 25.0 and 50.4 at G=1, 2, 4, 8, query time is close to linear in them wherever the table is resident, and the one granularity whose table does not fit issues the fewest of any and is slower regardless. The budget rule is worth 1.095–1.469× in steady state at S=128, repaid within 4.0 to 19.2 batches of 500 queries, while at a quarter of that sample the quality risk varies enough by workload that the default we started from does not survive its own test.

The competitive picture is bounded and we state it as such. Among quantised IVF methods JHQ leads through Recall@10=0.95 at large batch, owns the high-recall end at high dimension, and indexes two datasets where the tested

cuVS IVF-RaBitQ build path runs out of memory. It is not the fastest system measured: graph-based CAGRA-int8 is 1.6–2.9× faster below Recall@10=0.95 on every dataset where both run, at 3.9–7.4× the build time and a comparable code budget.

Contributions. (i) Evidence that the primary scan's cost divides into an issue term and a residency term: six controls in which fewer bytes or less state fails to predict throughput, and a static instruction count from the disassembled kernel against which the granularity whose table does not fit is slower than its instruction count alone can explain. **(ii)** An exact Cartesian factorisation of JHQ's primary distance table, derived from separability the codebook already has rather than from a different quantiser, with the regime in which it pays. **(iii)** A label-free policy for JHQ's residual budget, evaluated on gain, quality cost, amortisation and sampling risk against fixed budgets and a ground-truth reference. **(iv)** An evaluation that reports where the compared systems are faster than ours and where our evidence does not generalise.

2 Preliminaries

For a database vector x and query q in $\mathbb{R}^d$, JHQ applies an orthogonal $Q \in \mathbb{R}^{d\times d}$: $y = Qx$, $q' = Qq$ (row-major batches use $Y = XQ^\top$). Q is square, so this is a change of coordinates preserving Euclidean distance and hence the neighbour ordering, not dimensionality reduction. The transformed vector is split into M subspaces of dimension D_s, $d = MD_s$; each primary subspace code has B bits selecting one of $K = 2^B$ codewords, and each codeword is a Cartesian product of D_s one-dimensional Lloyd–Max levels. With L levels per coordinate $L = 2^{B/D_s}$, so the implementation admits only D_s dividing B: at B=8, $D_s \in \{1, 2, 4, 8\}$ gives $\{8, 4, 2, 1\}$ primary bits per coordinate. We use D_s=8, so each coordinate carries one primary bit and each subspace is one byte. This Cartesian construction is what Section 3.2 exploits.

The residual is $r = y - \hat{y}_{\text{primary}}$, the displacement from the primary reconstruction — *not* from an IVF centroid. IVF is an external routing layer that restricts the candidate set; it does not redefine what the residual quantiser represents. The residual code is per dimension, so at B_r bits it costs about dB_r bits per vector, and at B_r=8, D_s=8 the logical representation is about 1 primary plus 8 residual bits per dimension.

Routing yields a candidate set $C(q)$. The primary score D_P is evaluated for every routed candidate; JHQ keeps the c_k with the smallest primary scores and evaluates the hierarchical score D_H only for those, returning the top-k under D_H among them. We write $c_k = \alpha k$ for exposition; the implementation uses $c_k = \max(k, \lceil \alpha k \rceil)$, and since every evaluated $\alpha \geq 1$ the max only enforces the k-survivor floor. So α controls an irreversible filtering point: a neighbour dropped by primary ranking cannot be recovered by refinement, while routing can drop true neighbours before the primary stage at all. The two loss sources stay separate throughout.

3 GPU-Native JHQ

3.1 Execution and representation

We begin from the work JHQ requires after routing rather than from CUDA primitives. The primary stage is $O(n_c M)$ over compact codes and the residual stage $O(c_k d)$ over per-dimension codes, so three requirements follow: make primary access match candidate-parallel warp execution, keep the primary distance representation resident without replacing JHQ's quantiser, and preserve selective residual access instead of fusing accurate scoring into the broad scan. The orthogonal transform, primary codebook, residual definition and hierarchical ordering remain JHQ's; physical representation, execution and the external refinement policy change, and the scan uses a bounded per-thread top-αk selector whose deviation from exact selection Section 4.2 measures.

During the broad scan, warp lanes score different candidates while advancing through the same subspace m, so candidate-major $[N, M]$ storage has the warp read one byte per lane with stride M. We store the scan copy subspace-major, $[M, N]$, which makes those 32 bytes adjacent: the transpose aligns the contiguous memory dimension with the dimension the warp shares. It is standard practice, not a claim. Separately, $D_s{=}B{=}8$ makes four subspace codes a native 32-bit word that a packed scan loads once and unpacks in registers. After the layout fix the fetched sectors are already right, so packing reduces load instructions, addresses and loop iterations rather than logical bytes. We do not claim wider words are universally better; they add unpacking, dependencies and register pressure.

3.2 Cartesian factorisation of the primary table

The question is not how small the table can be but how much table state to materialise before saved residency is outweighed by added issued work. For subspace m and code c, let $T_m[c] = \|q'_m - C_m[c]\|_2^2$. Because $C_m[c]$ is a Cartesian product of one-dimensional levels and squared Euclidean distance is additive, any partition of the eight code decisions induces an exact sum of smaller tables. For the two-way partition, with $c_{\mathrm{hi}} = c \gg 4$ and $c_{\mathrm{lo}} = c \bmod 16$,

$$T_m[c] \;=\; T_m^{\mathrm{hi}}[c_{\mathrm{hi}}] + T_m^{\mathrm{lo}}[c_{\mathrm{lo}}]. \tag{1}$$

Each component holds 16 entries, so $G{=}2$ stores 32 per subspace and performs two lookups. Table construction falls by $16\times$, not the $8\times$ of resident entries: the full table evaluates $256D_s$ coordinate contributions per subspace and the split form $32(D_s/2) = 16D_s$. An equal G-way partition stores $G \cdot 2^{8/G}$ entries and needs G lookups per candidate–subspace, which states the trade-off analytically — state falls only until the table fits, while instruction demand keeps rising.

The residency target is narrower than the device's shared-memory limit, because the table is not the only claimant. The scan kernel's admission test is

$$\texttt{scan\_base} + M \cdot G \cdot 2^{8/G} \cdot 4 \;\le\; \texttt{smem\_optin}, \qquad \texttt{scan\_base} = 40 \cdot \texttt{BLOCK} \text{ bytes}, \tag{2}$$

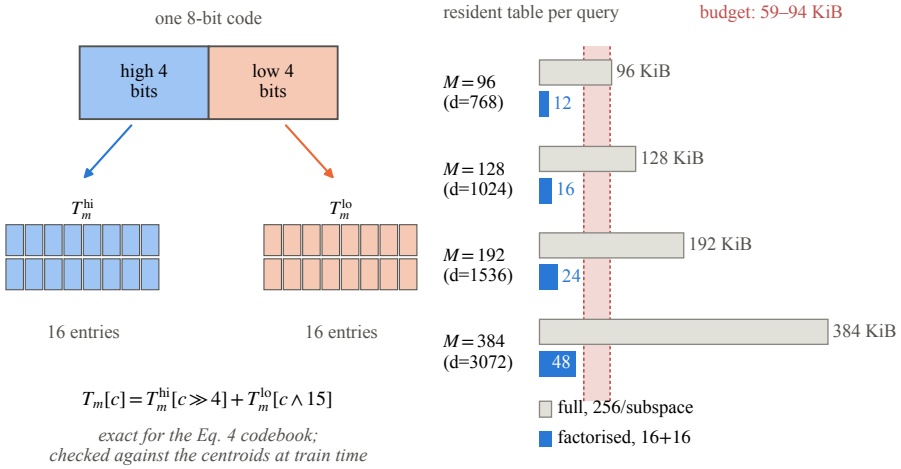

Fig. 1: Exact Cartesian factorisation of an 8-bit JHQ primary lookup table: 256 resident entries become two 16-entry tables. The band on the right is what Equation 2 leaves for the table across the admissible block sizes; every factorised table clears its lower edge and every full table overruns its upper one — M=96's by 2 KiB, the rest by more.

with `scan_base` the per-thread candidate scratch claimed first. At the tested `smem_optin` = 101,376 B this leaves the table 94.0 KiB at `BLOCK`=128, falling to 59.0 KiB at `BLOCK`=1024. The full table is $M \cdot 256 \cdot 4$ bytes — 96 KiB at M=96 and 384 KiB at M=384 — so *neither* clears the budget at any admissible block size, M=96 missing the most generous by 2 KiB, while both factorised tables clear the least generous at 12 and 48 KiB (Figure 1). The full table is thus off chip at both M, and what grows with M is the cost of the lookups that miss, since the scan pays one per candidate per subspace. The prediction we test in Section 6.3 is therefore directional: reducing table state should help until the extra lookup work outweighs the residency it buys, and the crossing should move with M and with probe depth together.

Grouped tables and the capacity-against-arithmetic trade-off are not new: GPU IVF-RaBitQ [8] groups U bits into 2^U-entry tables and picks U=4 on exactly that basis, and we claim neither. What Equation 1 adds is that the grouping exists at all here — JHQ's 8-bit code indexes 256 codewords with no grouped structure on its face, and the split follows from the Cartesian construction rather than from choosing a different quantiser. It does not exist for an ordinary k-means product quantiser at the same width. Exactness is in real arithmetic; it promises no bitwise-identical floating-point association or tie order.

3.3 Selective residual refinement, and what it leaves open

Reading the per-dimension residual for every routed candidate would turn an $O(c_k d)$ stage into work closer to $O(n_c d)$, so only the c_k primary survivors trigger

residual access. Fusing the two phases is a plausible alternative — it removes a round trip and a kernel boundary — and Section 6.4 reports why we reject it.

4 Adaptive Hierarchical Refinement

4.1 Criterion

Original JHQ leaves α externally specified, and the useful budget is workload dependent: at *nprobe*=128 openai3-3072 is flat from α=4 over the measured grid while arxiv-768 is still improving at α=200, a censored upper endpoint rather than a plateau. A single global constant therefore either wastes $O(\alpha kd)$ residual work on easy workloads or filters candidates before accurate refinement on hard ones. We calibrate a workload-level operating point rather than predict α per query, which keeps the control path label-free and amortisable.

Let Q_s be S sampled queries from the current workload and $R_{\max}(q)$ the top-k ID set returned under a generous $\alpha_{\max}$. For a tested α with returned set $R_\alpha(q)$, define the total slot disagreement

$$D(\alpha) \;=\; \sum_{q \in Q_s} \big[\, k - |R_\alpha(q) \cap R_{\max}(q)| \,\big], \tag{3}$$

and select the smallest tested-grid α with $D(\alpha) \le \epsilon$. The tolerance ϵ is a non-negative integer over the whole sample, not a per-query fraction. Set membership asks whether more residual work changes *which* neighbours are returned while ignoring harmless permutations of the same IDs. S=32 and ϵ=1 were our initial defaults; Section 6.5 measures what they risk and withdraws the first of them. The two are also not independent: ϵ counts slots over the whole sample, so a fixed ϵ is a stricter test as S grows. Ground-truth recall is what we care about but is unavailable online, and a learned predictor costs labels, maintenance and drift behaviour outside this scope. Output stability is observable from the running index, but cannot recover routing losses, certify recall, or see past $\alpha_{\max}$; Section 6.5 validates it against ground truth offline rather than treating self-agreement as a guarantee.

4.2 Monotonicity and bisection

Proposition 1 (monotonicity of disagreement). *Let $\alpha_1 \le \alpha_2 \le \alpha_{\max}$. Under exact top-$c_k$ primary selection with c_k non-decreasing in α, a fixed refined distance, and a deterministic total tie rule, $D(\alpha_1) \ge D(\alpha_2) \ge D(\alpha_{\max}) = 0$.*

Proof. Exact top-c_k selection nests the survivor sets, $S_{\alpha_1} \subseteq S_{\alpha_2} \subseteq S_{\alpha_{\max}}$. Order $S_{\alpha_{\max}}$ by the refined distance with the deterministic tie key and let $R_{\max}$ be its first k elements. Any element of $R_{\max}$ present in S_α also appears in top-$k(S_\alpha)$, since fewer than k elements of $S_{\alpha_{\max}}$ precede it and hence fewer than k of the subset S_α do. So $R_\alpha \cap R_{\max} = S_\alpha \cap R_{\max}$, which can only grow with α. □

Implementation caveat and measured deviation. The production scan uses a bounded per-thread selector with `K_LOCAL`=4, so a thread can discard a globally top-c_k item when more than `K_LOCAL` of them fall in its stride class, and theorem-level nestedness is not asserted for the deployed kernel. We measure the gap rather than hide it behind aggregate recall: with the index pinned, vogue-768 at *nprobe*=128 and `BLOCK`=512, raising `K_LOCAL` from 4 to 8 changes 10 of 10,000 returned ID positions across 5 of 1,000 queries, and *no* query gets a different result set: every difference is two adjacent ranks swapping inside an identical top-10, so here the bounded selector reorders ties rather than discarding. Which items tie moves with the codebook, so this is one index instance, not a constant. `BLOCK`=512 is deliberately the harder collision regime, and `K_LOCAL`=8 is register-infeasible at the production `BLOCK`=1024.

The consequence for bisection should be stated precisely. Across all 15 calibration traces that probe two or more grid points, $D(\alpha)$ is non-increasing at the points the search visited. That is evidence, not a proof: points the search never visited are untested, so we do not claim bisection returns the global minimum acceptable α on the grid, only that its stopping behaviour is conservative with respect to the sampled criterion. Floating-point ties are a separate matter — the proposition assumes a deterministic total order, not bitwise-identical association.

Sampled parameter tuning is not itself new: FAISS [5] exposes parameter-space exploration and Li et al. [6] learn adaptive termination. Our contribution is narrower — a predictor-free top-k stability signal applied to JHQ's external residual budget, with its proxy error and economics quantified.

4.3 Calibration economics

With T_{cal} the calibration time and T_{fixed}, T_{rule} the per-batch times before and after selection, the break-even batch count is $B^* = T_{\text{cal}}/(T_{\text{fixed}} - T_{\text{rule}})$ whenever $T_{\text{rule}} < T_{\text{fixed}}$. A more expensive calibration is rational only when the budget saves enough steady-state work and the operating point persists; Section 6.5 reports both terms. Reuse assumes the query distribution, routing configuration, k and index stay stable — the price of workload-level rather than per-query adaptation. Drift is unmeasured: recalibrating every H batches adds T_{cal}/H per batch, but choosing H is a drift-detection problem outside this evidence. The timer starts after sample copying, so the cost is algorithm-level, not a full control-plane cost.

5 Related Work

We organise related work around the novelty threats that matter for this paper rather than as a catalogue.

Quantisation and JHQ. Product Quantization (PQ) [4] established Cartesian decomposition across subspaces and asymmetric distance computation. JHQ [3]

targets high-dimensional ANN with an orthogonal transform, a Cartesian primary quantiser, and hierarchical residual refinement. We do not claim JHQ's hierarchy or orthogonal transform as contributions; our work begins from that representation and asks how its specific structure should execute on a GPU.

Grouped lookup tables. This is the closest prior work and we state the overlap plainly. PQ Fast Scan [1] showed that cache residency alone is insufficient for fast ADC; Quicker ADC [2] generalised it with irregular product quantisers and split tables. Most directly, GPU IVF-RaBitQ [8] partitions dimensions into groups of U bits, builds one 2^U-entry table per group, sums D/U lookups per candidate, and selects $U{=}4$ by trading shared-memory capacity against arithmetic. That trade-off is the same one Section 3.2 navigates, it is prior art, and we do not claim it.

Two things are ours. First, the object: RaBitQ groups a representation designed to be grouped — its code *is* the bit pattern its table is indexed by — whereas JHQ's primary code is an 8-bit index into 256 codewords with no grouped structure on its face. Equation 1 is an identity that has to be derived from the codebook's Cartesian construction, and it does not exist for an ordinary k-means product quantiser at the same code width. Second, the method: RaBitQ selects its group size by measuring which performs best, while Section 6.3 reads an instruction count off the disassembled kernel and uses it to say which of the two competing costs each granularity is paying.

The two choices agree, and it is worth saying so rather than manufacturing a difference. RaBitQ's U counts bits per group and our G counts groups of an eight-bit code, so $U{=}8/G$ and its $U{=}4$ is our $G{=}2$: both land on four bits and sixteen entries per table. Two derivations over different representations reaching the same group width is corroboration that the trade-off is real, not evidence that either is novel for choosing it.

Adaptive ANN control. ANN libraries such as FAISS [5] commonly expose parameter sweeps and tuning utilities, while learned approaches can adapt search termination to the query [6]. We therefore do not claim sampling or parameter selection as new. Our output-stability rule differs in its signal and deployment assumptions: it needs no labelled nearest neighbours and no learned predictor, compares the system's own returned top-k sets to a generous-budget reference, and chooses the externally specified JHQ refinement budget. The evaluation separately quantifies selection error and amortisation, which are necessary because a label-free proxy is not a recall guarantee.

GPU ANN systems. CAGRA [7] represents the graph-based GPU frontier, and cuVS provides production implementations of CAGRA, IVF-PQ, and IVF-RaBitQ. GPU IVF-RaBitQ [8] combines inverted-file routing with low-bit RaBitQ codes and reports a strong recall–throughput/build/storage trade-off. These systems are not uniformly weaker than JHQ: our matched experiments show regime crossings in the high-recall tail and with batch size. This is why we re-

port nondominated frontiers and configuration-specific build failures rather than a single aggregate speedup.

6 Evaluation

6.1 Setup and protocol

All GPU systems run on one NVIDIA RTX 5090: 32,607 MiB device memory, 170 SMs, a 101,376-byte opt-in shared-memory limit per block; the host has 208 logical cores and 754 GiB RAM. Datasets are vogue-768 (1.0M $\times$ 768), arxiv-768 (2.25M$\times$768), bge-m3 (10.1M$\times$1024), stella-trec24 (17.8M$\times$1024), openai3-1536 (1.0M $\times$ 1536) and openai3-3072 (1.0M $\times$ 3072).

We report Recall@10 against true top-10 ground truth at k=10; because c_k scales with αk, other k are not established. Timing runs host query input to host result output, and frontiers use steady-state throughput at batch 1,024 — the whole query set in one call — which Section 6.6 shows is a consequential choice. Occupancy statements are analytical resource-budget calculations: Nsight Compute counters were unavailable in the container (`ERR_NVGPUCTRPERM`), so we report none measured. Two recall noise floors are kept distinct: about 10^{-3} across builds in the documented cold-start experiment, and $\approx 10^{-4}$ for paired comparisons on one index from top-c_k tie breaking. The CPU comparison buckets recall at 10^{-3} so a tie-break-sized difference cannot buy a point a place; the GPU and baseline fronts are plain Pareto envelopes over the measured points, and the cuVS columns are the harness's three-run mean throughput, not a median. We do not enlarge batches by duplicating the roughly 1k available queries: duplication alone raises throughput 4–26%, so such a batch would measure an artefact.

Baselines are the cuVS implementations of IVF-RaBitQ [8], IVF-PQ, and CAGRA [7] in fp32 and int8. Our primary comparison line is the quantised IVF family, which shares JHQ's routing structure and interface; CAGRA is a graph index with a different build-cost profile and we report it as context, in both variants, without excluding the one that wins. JHQ runs use dense coarse training, `JHQ_N_TRAIN` $= 39 \cdot$ *nlist*; we mix in no under-trained points and union no fronts across binaries. The CPU JHQ baseline runs the same index parameters on the same host with threads pinned (`OMP_PROC_BIND=close`, `OMP_PLACES=cores`) and its own (α, *nprobe*) sweep, sharing the training procedure but not a codebook, with one measurement per cell.

6.2 RQ1: where does JHQ actually lead?

Figure 2 gives the measured frontiers and Table 1 reduces them to a single question: at each matched recall, which system is fastest? JHQ owns 4 of the 25 cells, and all four are at Recall@10 $\geq$ 0.97. CAGRA-int8 leads in 13 of them, by 1.3–2.9$\times$ and concentrated below Recall@10=0.95 — 1.6–2.9$\times$ over the cells strictly below it; CAGRA-fp32 leads in 3 by 1.1–1.4$\times$ and IVF-RaBitQ in 5 by 1.0–1.4$\times$, measured on the 4 of those 5 JHQ also reaches. One row is

Table 1: Fastest system at each matched recall, batch 1,024, with its margin over JHQ. int8 and fp32 are cuVS CAGRA, RaBitQ is cuVS IVF-RaBitQ; "–" is a recall no two systems both reach. JHQ is fastest in 4 of 25 cells, all at Recall@10 ≥ 0.97.

	fastest system at Recall@10					
dataset	0.90	0.93	0.95	0.97	0.98	0.99
vogue-768	int8 2.7×	int8 2.0×	fp32 1.1×	RaBitQ 1.0×	RaBitQ 1.1×	**JHQ**
arxiv-768	int8 2.9×	int8 2.9×	int8 2.5×	fp32 1.4×	fp32 1.4×	RaBitQ
bge-m3	int8 2.4×	int8 1.7×	–	–	–	–
stella-trec24	–	–	–	–	–	–
openai3-1536	int8 1.9×	int8 1.6×	int8 1.3×	**JHQ**	**JHQ**	–
openai3-3072	int8 2.4×	int8 2.1×	int8 1.6×	**JHQ**	RaBitQ 1.1×	RaBitQ 1.4×

excluded throughout: stella-trec24's CAGRA-int8 point at Recall@10 of 0.9692 is `status=CONTAMINATED` in the harness's own record — a 6.6% throughput spread against 0.02–0.64% elsewhere — and the frontier generator had dropped only `FAILED` rows. We state this rather than scope it away, because it bounds the claim precisely: JHQ's region is the high-recall end, most clearly at high dimension, where openai3-1536 leads at 0.97 and 0.98 and openai3-3072 at 0.97.

Two things beside the table complete the picture. On bge-m3 and stella-trec24 the tested cuVS IVF-RaBitQ host-input build path fails with an out-of-memory allocation after entering streaming construction and CAGRA fp32 does not fit the card; on stella-trec24 IVF-PQ fails the same way, which is why Figure 2 shows it one curve short of bge-m3. JHQ indexes all six — a statement about the measured library and configuration paths on this device, not about those algorithms in general. And the throughput ordering has a build-time counterpart. Comparing cold builds per dataset, JHQ takes 1.1–13.4 s against 4.9–74.3 s for CAGRA int8, between 3.9× and 7.4× faster. Stated as a crossover rather than a ratio: where CAGRA int8 is the faster searcher, its throughput repays the build gap after 1.6M to 8.9M queries across the datasets and the two recall targets. Below that volume JHQ costs less end to end, and we use CAGRA's fastest observed build so the crossover is the one least favourable to us. JHQ therefore trades search throughput below 0.95 for build time and for indexability at a code budget — about 9 bits per dimension — that is not smaller than int8's.

6.3 RQ2: does the design prediction hold?

Section 3.2 predicted that reducing table state helps only until extra lookups outweigh the residency saved. Figure 3 varies G with the represented distance fixed. If state alone governed throughput, G=4 and G=8 should win, since they store fewer entries than G=2. They do not: G=2 wins all eight configurations in Figure 3(a) and throughput falls as lookups per candidate rise. Its narrowest win is where the effect stops being resolvable: a second phase put vogue-768 at *nprobe*=32 in the packed layout at −4.2% where this one puts it at +4.2%, and one measurement a side cannot separate a 4% effect from run noise — which is

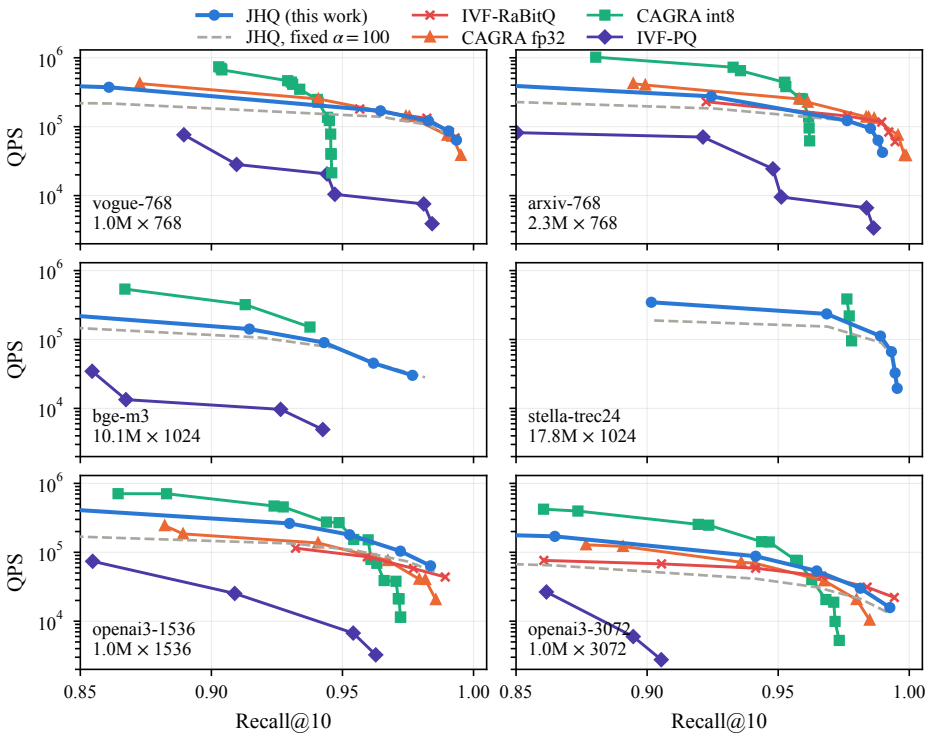

Fig. 2: Measured Recall@10–QPS frontiers, batch 1,024. Four datasets carry complete JHQ/IVF-RaBitQ curves; the two largest show the baselines that could be built and searched on the tested card. Sweep endpoints are measured endpoints, not architectural recall ceilings.

why the design is stated as a regime rather than a rule. The sharpest form is internal to the losers. $G \cdot 2^{8/G}$ is 16 for both G=4 and G=8, so they occupy the same nominal table footprint and differ in the number of lookups and the address arithmetic those lookups need; the eight-way form is 5.8% slower at *nprobe*=8 and 38.3% slower at 512. Residency is held fixed and issued work rises — which is why this pair, rather than the table-free control, separates the two effects most cleanly.

The regime change with M appears as predicted. Against the full table, factorisation is worth +0.7–+10.5% at M=96 and +8.3–+53.8% at M=384, and at M=384 the gain rises monotonically with probe depth: on openai3-3072 +8.3%, +10.1%, +22.7% and +53.8% at $nprobe = 8, 32, 128, 512$. We report the trend rather than a flat interval because the trend is the mechanism — a table that misses is paid once per candidate per subspace, so its cost grows with M and with the candidates scanned together. At M=96 it does not rise monotonically — +3.6, +4.2, +0.7, +10.5% — which restates the −4.2% above: at M=96 the full table misses the carveout by 2 KiB against 290 KiB at M=384, so little residency is left to win back.

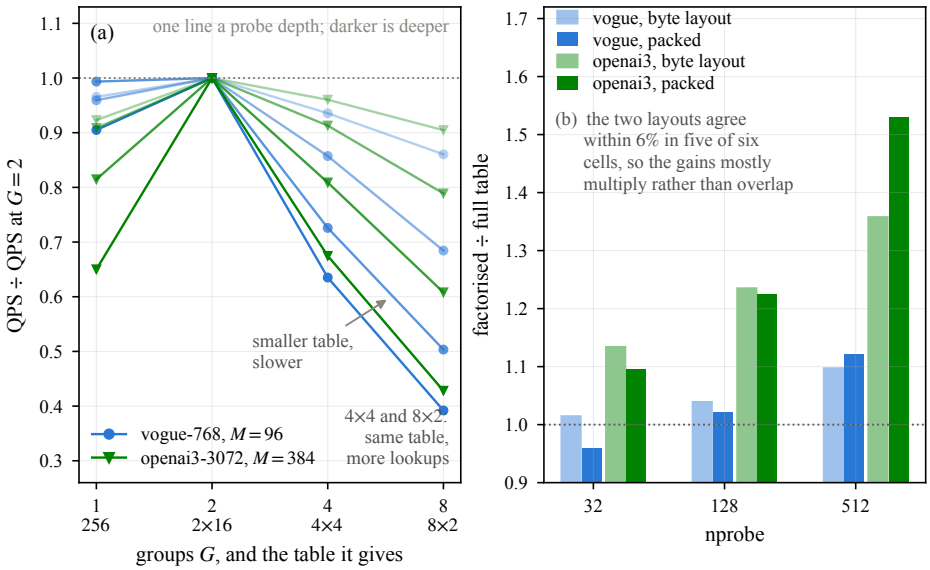

Fig. 3: (a) G=2 wins all eight configurations of the granularity sweep although G=4 and G=8 store fewer entries and occupy the same footprint as each other; every line is one probe depth, normalised to its own G=2. (b) Factorisation matters far more at M=384 than M=96 and stays largely complementary to packed loading. The two panels are different phases of the run — (a) the granularity sweep at *nprobe* $\in \{8, 32, 128, 512\}$, (b) the two-by-two table×layout phase at *nprobe* $\in \{32, 128, 512\}$ — and are counted separately.

6.4 RQ3: can the byte-count explanation be rejected?

Table 2 collects the controls that vary bytes and issued work independently. Removing almost all table state loses 30–52% while *raising* the resident-block count the shared-memory budget implies, so neither residency nor attainable occupancy alone is the bottleneck; moving identical bytes in 32-bit words gains 1–66% across six datasets, rising from 1–6% at *nprobe*=8 to 9–66% at 512 — a load is paid once per candidate per subspace, so its share grows with the candidates scanned together, and a flat interval would hide that; fusing residual refinement removes a round trip and still loses, the fused kernel carrying both phases' footprints for the whole scan. The probe cursor is an implementation correction rather than a contribution, included because its size is diagnostic. LUT construction is 0.2–2.3% of a batch, which rules out faster table building as the explanation for factorisation.

These controls reject competing explanations but do not measure the variable they implicate, and static instruction counts need no profiler. `cuobjdump -sass` on `scan_ivf_coalesced_kernel` reports 42, 84 and 168 generic loads at G=1, 2, 4 — G lookups across 42 statically unrolled subspace iterations — and 144 at G=8, eight lookups over 18 iterations after the compiler unrolled less. Floating-point adds are G+1 per iteration throughout, which checks the

Table 2: Controls that separate bytes moved from work issued. Every row holds the represented distance fixed.

change	effect on throughput	what it rules out
table-free exact distance	−30 to −52%	least state is fastest
G=8 against G=4	−5.8 to −38.3%	same footprint, same speed
packed 32-bit loads	+1 to +66%	bytes moved predict speed
private probe cursor	−1.4 to +148.5%	loop work is negligible
fused residual refinement	−14.2%, −14.8%	fewer kernels is better
query duplication D=8	+4 to +26%	reuse alone drives the scan

normalisation. Counting the lookup and what feeds it gives 5.0, 13.0, 25.0 and 50.4 static instructions per candidate–subspace. These are static counts over the kernel's disassembled span, not a dynamic issue count.

One statement then needs no model: G=1 *issues* 5.0 *instructions per candidate–subspace against* G=2*'s* 13.0*, and is slower at most operating points anyway.* It issues the fewest of any granularity while holding the most table state — 256 entries a subspace against 32 — so instruction count predicts it should be fastest and it is not, and the state it holds is the one thing that separates it. Figure 4 puts numbers on that, and they are an extrapolation: a least-squares line through the three resident points, extended back to G=1's count, leaves its measured time 6, 14, 23 and 56% high on vogue-768 and 10, 16, 40 and 108% high on openai3-3072 at $nprobe = 8, 32, 128, 512$ — from 5.0, outside a fitted range of 13.0 to 50.4. We read the sign and the trend: the shortfall grows with probe depth and with M, as "a table that misses is paid once per candidate per subspace" predicts, and G=2 pays least of either cost. Instruction counts and controlled comparisons together are consistent with this account on the tested card; they are not an independent measurement of the two terms, which would need a build holding residency fixed while sweeping instruction count alone.

6.5 RQ4: would one fixed budget do instead?

Five arms share one index and one timing harness, calibrating on even-indexed queries and reporting every recall on the odd-indexed half: each fixed α on the grid; the sampled rule over 64 random samples at each S; the same criterion on the whole calibration pool; a whole-grid walk on the sample the rule bisects; and, for reference only, the fastest α whose held-out recall is within 10^{-3} of the best any α reaches, chosen with ground truth.

A constant meets the quality bar only by overpaying. The reference budget is α=64 on vogue-768 at both probe depths and α=8 on openai3-3072, an order of magnitude apart. On openai3-3072 α=4 ties it to within 0.1% and 2.6% of throughput at equal recall and a repeat run selects it instead, so the measurement resolves the magnitude there, not the grid point. The conservative end generalises — α=64 stays within 10^{-3} of attainable recall in all four cells — but on openai3-3072 the rule's budget runs 32% faster at *nprobe*=128 and 7% faster at 512, for equal recall at the first and 0.0002 less at the second. The cheap end does not

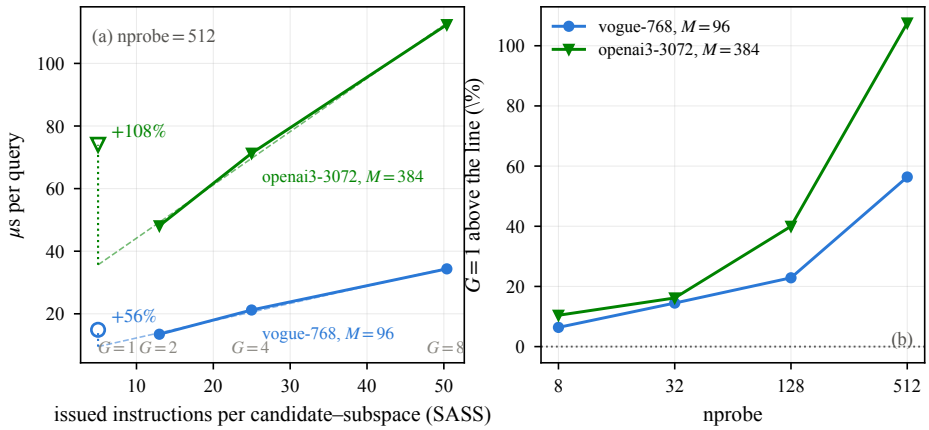

Fig. 4: Instruction count against measured time. (a) With the table resident query time is close to linear in issued instructions per candidate–subspace; G=1 issues the fewest of all, holds the most table state, and sits above a line fitted to the other three and extrapolated beyond its range — consistent with a residency cost. The dashed line, the annotation and the text are the same least-squares fit. (b) That shortfall grows with probe depth and with M.

generalise at all: α=8 gives up 0.0366 of recall on vogue-768, and α=4 0.0884. The question is therefore what a fixed conservative budget costs, which the earlier comparison against α=100 could not address.

The rule finds it, given enough sample. The median selection over 64 random samples matches the reference on vogue-768 at $S \geq 64$ and lands on the tied α=4 on openai3-3072; at S=32 it picks 32 where the reference is 64 on vogue-768. Against the α=100 default the selection is worth 1.095–1.469× in steady state, repaid in 4.0 to 19.2 batches.

The sample is the risk, and S=32 is too small. Across the four cells, 25–69% of random samples at S=32 give up more than 10^{-3} of held-out recall, with a 95th percentile of 0.0116 on vogue-768. At S=64 that is 2–45% and at S=128 0–8% — still 5 of 64 samples on vogue-768 at *nprobe*=128. We withdraw Section 4's S=32: S is a tolerance decision, and S=128 is where the tolerance becomes small, not where it vanishes. Figure 5(b) resamples that risk 2,000 times per cell and shows it is workload dependent, not a constant of S.

Neither the sample nor the bisection is the approximation. Walking the whole grid on the sample the rule bisects reproduces its choice in all 768 draws at 1.5–1.7× the cost. The same criterion on the entire calibration pool is 2.1–2.9× more expensive and selects a *more* conservative budget, α=100 where the reference is 64 on vogue-768. The cause is in the criterion's definition: ϵ is an absolute count of disagreeing slots, and a sample of S queries has $S \cdot k$ of them, so the same ϵ=1 admits one miss in 320 slots at S=32 and one in 5,000 at the full pool. The tolerance tightens as the sample grows, which is a property worth stating plainly — ϵ and S cannot be chosen independently.

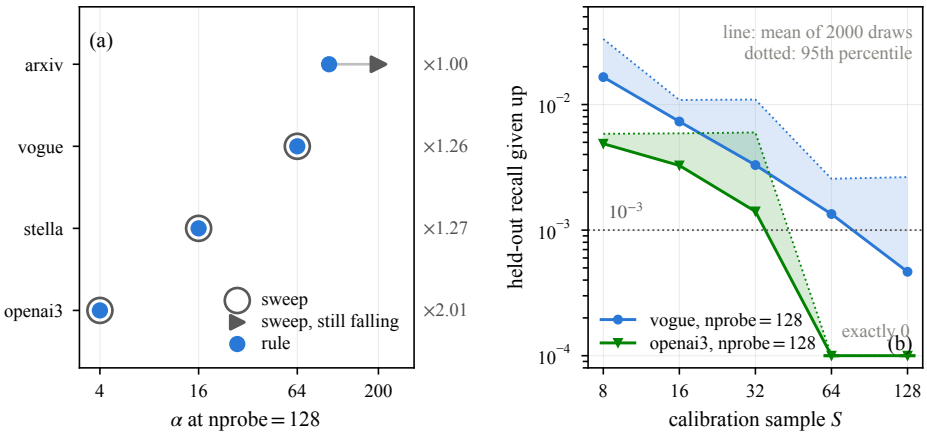

Fig. 5: Calibration validation. (a) Selected α against the exhaustive sweep where a same-operating-point sweep exists, annotated with the throughput the selection gains; a hollow ring around a marker is a sweep that agrees with the rule, and the arrow is a sweep still improving at $\alpha_{\max}$. (b) Held-out resampling over 2,000 draws, a larger resampling than this section's 64, showing that sample-size risk is workload dependent.

Two earlier observations survive. The rule recovers the swept α in three of four configurations at *nprobe*=128 (Figure 5(a)), the fourth censored by $\alpha_{\max}$; and over a broader 32-configuration sweep against α=100 — six datasets at S=32, a different configuration from this section's, which we do not combine with it — gain is 1.044–2.653× for at most 0.0048 recall, median payback 1.75 batches.

6.6 RQ5: batch size and the CPU reference

Table 3 gives matched-recall ratios against IVF-RaBitQ at four batch sizes. Two readings matter: on openai3-3072 JHQ leads at every measured point, and on vogue-768 it crosses parity between batch 128 and 512. Batch size can therefore change the ordering, and the frontier ratios of Section 6.2 should be read as the batch-1,024 column of this sweep rather than as batch-independent summaries. The openai3-3072 batch-1,024, R=0.97 cell is excluded: its RaBitQ anchor is the one matched point that failed the project's monotonicity check.

The CPU reference answers a different question: what the port buys over CPU JHQ at the same index parameters. Sweeping α and *nprobe* on both sides, the GPU reaches 52×, 53× and 85× the CPU envelope on vogue-768 at Recall@10 of 0.9645, 0.9828 and 0.9905, and 75–77× on openai3-3072 from 0.9414 to 0.9813. Pinning α=100 on *both* sides gives 41×, 45× and 71× on vogue-768 and 48–60× on openai3-3072 — the two move in opposite directions because the envelope's freedom over α and threads is worth more to the CPU on openai3-3072. The vogue figure rises steeply at the top because the CPU degrades faster than the GPU as *nprobe* grows, and its highest point rests on the noisiest CPU

Table 3: Matched-recall JHQ / IVF-RaBitQ throughput, by batch size, at Recall@10 of 0.90 and 0.95. Each system's curve is interpolated at the target recall.

	openai3-3072		vogue-768	
batch	0.90	0.95	0.90	0.95
32	2.23	1.34	0.68	0.55
128	3.19	2.14	0.99	0.92
512	2.30	1.51	1.06	1.01
1024	1.89	1.41	1.09	1.33

cell we measured — at *nprobe*=1024 and 32 threads the six α settings span 792 to 1,072 QPS — so we quote it as a point rather than fold it into a range. Five repeats of that cell put its two relevant α at medians of 1,027 and 1,003 QPS, the latter swinging 765 to 1,015 against 2% on its neighbour; the ratios above use those medians, and the single-pass log gave 90× and 80×. Above Recall@10 of 0.9911 and 0.9900 the retained envelope has no points (openai3-3072's raw α=100 rows reach 0.9904, but are dominated), and GPU points above that are reported as outside its range rather than divided by the CPU's last measurement. Three limits apply. The GPU side of the vogue comparison is the rule's chosen α at each *nprobe*, not an exhaustive α sweep. CPU rows below *nprobe*=128 are dropped, because 1,000 queries complete fast enough there that thread start-up dominates the timer and measured throughput rises with *nprobe*; the recall column is unaffected. And each cell is measured once except the one just described, so the ratios are order-of-magnitude figures — the envelope keeps the fastest point per recall bucket, which biases the CPU side upward and so does not inflate the ratio, but with one measurement per cell we do not claim a statistical bound.

6.7 Construction and memory

Cold build times are 1.1–13.4 s for JHQ, 4.9–74.3 s for CAGRA int8, 1.9–39.0 s for IVF-PQ and 4.3–13.0 s for IVF-RaBitQ where that path builds — each baseline's fastest build per dataset, ranged across datasets, which is the choice least favourable to us; tested configurations, not intrinsic bounds. JHQ's figure is the cold-cache run at *nprobe*=8; its coarse training is cached in later runs, so a warm build is several times cheaper and is not what we report. Accounting must respect caching: training can be cached while add is not, so taking maxima independently and summing them fabricates a time that never occurred. On memory we claim no advantage: in the four datasets with measured VRAM JHQ uses 22–40% more than IVF-RaBitQ, and its budget is not smaller than int8's. JHQ occupies a different memory–accuracy–throughput point and still indexes the two largest datasets where the compared paths fail. The accounting carries an unattributed component that should not be relabelled as allocator overhead without direct measurement.

7 Conclusion

We asked how JHQ's primary representation should execute on a GPU and how much of its hierarchy is worth executing. For the first, the Cartesian codebook admits an exact two-table factorisation whose state grows as $32M$ against $256M$, and the measurements show the payoff rising with both M and probe depth — -4 to $+12\%$ at M=96 against $+8$ to $+54\%$ at M=384 — while still-smaller splits lose to their extra lookups. Disassembling the kernel adds the term the controls could not reach: the granularity whose table does not fit issues the fewest instructions per candidate–subspace and is slower anyway, so the target is a residency budget rather than minimum bytes. For the second, a label-free stability rule buys 1.095–1.469× at S=128, repaid within 4.0 to 19.2 batches of 500 queries, and its median choice over 64 samples matches the offline reference budget in all four measured cells — but 5 of 64 samples still exceed a 10^{-3} loss on one of them, so the sample size is a risk decision, not a default. The region this defines is bounded: among quantised IVF methods JHQ leads to Recall@10=0.95 at large batch and owns the high-recall end at high dimension, while CAGRA-int8 is faster below that at several times the build cost. Remaining gaps are k beyond 10, workload drift, and cross-GPU validation.

References

1. André, F., Kermarrec, A.M., Scouarnec, N.L.: Cache locality is not enough: High-performance nearest neighbor search with product quantization fast scan. Proc. VLDB Endow. **9**(4), 288–299 (2015). https://doi.org/10.14778/2856318.2856324
2. André, F., Kermarrec, A.M., Scouarnec, N.L.: Quicker ADC: Unlocking the hidden potential of product quantization with SIMD. IEEE Trans. Pattern Anal. Mach. Intell. **43**(5), 1666–1677 (2021). https://doi.org/10.1109/TPAMI.2019.2952606
3. Han, J., Zhang, M., Trajcevski, G.: JHQ: Johnson-Lindenstrauss enhanced hierarchical quantization for high-dimensional approximate nearest neighbor search. Proc. VLDB Endow. **19**(7), 1530–1543 (2026). https://doi.org/10.14778/3801059.3801067
4. Jégou, H., Douze, M., Schmid, C.: Product quantization for nearest neighbor search. IEEE Trans. Pattern Anal. Mach. Intell. **33**(1), 117–128 (2011). https://doi.org/10.1109/TPAMI.2010.57
5. Johnson, J., Douze, M., Jégou, H.: Billion-scale similarity search with GPUs. arXiv:1702.08734 (2017)
6. Li, C., Zhang, M., Andersen, D.G., He, Y.: Improving approximate nearest neighbor search through learned adaptive early termination. In: SIGMOD. pp. 2539–2554 (2020). https://doi.org/10.1145/3318464.3380600
7. Ootomo, H., Naruse, A., Nolet, C., Wang, R., Feher, T., Wang, Y.: CAGRA: Highly parallel graph construction and approximate nearest neighbor search for GPUs. In: IEEE ICDE (2024). https://doi.org/10.1109/ICDE60146.2024.00323
8. Shi, J., Gao, J., Xia, J., Feher, T.B., Long, C.: GPU-native approximate nearest neighbor search with IVF-RaBitQ: Fast index build and search. Proc. VLDB Endow. **19**(11), 3454–3467 (2026). https://doi.org/10.14778/3836663.3836701